📘 Appunti — terza-app-spring-H2

Panoramica del Progetto

Questa è una **REST API** costruita con Spring Boot che gestisce uno scenario universitario semplice: **Studenti** (`Student`) e **Corsi** (`Course`) con una relazione **molti-a-molti** tra loro. Il database usato è **H2** (in-memory), ideale per sviluppo e testing. La struttura segue il classico pattern a strati di Spring: **Entity** → **Repository** → **Service** → **Controller**.

Struttura del Progetto

Il package radice è `com.dario.webapp` e contiene quattro sottopacchetti:

```
com.dario.webapp
├── entity/ → le classi che mappano le tabelle del DB
├── repository/ → interfacce per accedere al DB
├── service/ → logica di business
└── controller/ → endpoint REST esposti all'esterno
```

Passo 1 — Le Entity

Le entity sono le classi Java che JPA/Hibernate trasformano in tabelle del database.

`Student.java`

La classe `Student` rappresenta uno studente con i seguenti campi:

```
@Entity
@Table(name = "students")
public class Student {
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false)
private String firstName;
@Column(nullable = false)
private String lastName;
@Column(nullable = false)
private String email;

@ManyToMany
@JoinTable(
    name = "students_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id")
)
@JsonIgnore
private Set<Course> courses = new HashSet<>();
...
```

}

`Course.java`

La classe `Course` rappresenta un corso universitario:

@Entity
@Table(name = "courses")
public class Course {

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false, unique = true)
private String title;
```

```
private String description;

@ManyToMany(mappedBy = "courses")
@JsonIgnore
private Set<Student> students = new HashSet<>();
...
```

}

Passo 2 — I Repository

I repository sono **interfacce** (non classi concrete) che estendono JpaRepository. Come scritto nel codice stesso: *"In questo caso vado ad estendere JpaRepository così non devo fare DAO manuali. Inoltre seguo il pattern architetturale di Spring vero e proprio"*.

```
public interface StudentRepository extends JpaRepository<Student, Long> {
boolean existsByEmail(String email);
}
```

Estendendo `JpaRepository<Student, Long>` si ereditano automaticamente metodi come:

- `findAll()` — recupera tutti gli studenti
- `findById(Long id)` — recupera uno studente per ID
- `save(Student student)` — salva o aggiorna uno studente
- `deleteById(Long id)` — elimina uno studente

Il metodo `existsByEmail` è un **metodo derivato**: Spring capisce dalla firma che deve eseguire una query `SELECT ... WHERE email = ?` per controllare se esiste già un'email.

Passo 3 — I Service

I service contengono la **logica di business**. Come scritto nel codice: *"Nei service vado ad implementare le logiche di business legate alle singole entities"*.

La classe `StudentService` è annotata con:

```
@Service
@Transactional
public class StudentService {
// logica di business
}
```

`StudentService` usa entrambi i repository (sia `StudentRepository` che CourseRepository) perché la logica di enrollment coinvolge due entity. Il metodo chiave è:

```
public Student enrollStudentInCourse(Long studentId, Long courseId)
{
Student student = findStudentById(studentId);
Course course = courseRepository.findById(courseId)
.orElseThrow(() -> new RuntimeException("Corso non esistente con
questo ID"));
student.addCourse(course);
return studentRepository.save(student);
}
```

Questo metodo:

1. Trova lo studente per ID
2. Trova il corso per ID
3. Chiama `addCourse()` che aggiorna **entrambi i lati** della relazione ManyToMany
4. Salva lo studente nel DB

Passo 4 — I Controller

I controller espongono gli **endpoint HTTP REST**. Usano i service (mai i repository direttamente) e sono annotati con `@RestController`:

```
@RestController
@RequestMapping("/api/students")
public class StudentController {
// endpoint
}
```

Endpoint Studenti (`/api/students`)

Metodo HTTP	Endpoint	Descrizione
GET	/api/students	Recupera tutti gli studenti
GET	/api/students/{id}	Recupera uno studente per ID
POST	/api/students	Crea un nuovo studente
PUT	/api/students/{id}	Aggiorna i dati di uno studente
DELETE	/api/students/{id}	Elimina uno studente
POST	/api/students/{studentId}/enroll/{courseId}	Iscrivi uno studente a un corso

Endpoint Corsi (`/api/courses`)

Metodo HTTP	Endpoint	Descrizione
GET	/api/courses	Recupera tutti i corsi
GET	/api/courses/{id}	Recupera un corso per ID
POST	/api/courses	Crea un nuovo corso
PUT	/api/courses/{id}	Aggiorna i dati di un corso
DELETE	/api/courses/{id}	Elimina un corso

Dizionario delle Annotation

Annotation di Classe

Annotation	A cosa serve
`@Entity`	Dice a JPA che questa classe Java è una tabella nel DB
`@Table(name="...")`	Specifica il nome esatto della tabella nel DB
`@RestController`	Combina @Controller + @ResponseBody: tutti i metodi restituiscono JSON automaticamente
`@RequestMapping`	Definisce il prefisso URL dell'intero controller
`@Service`	Marca la classe come componente di business logic
`@Transactional`	Spring gestisce automaticamente le transazioni DB — commit se OK, rollback se errore

Annotation di Campo (Entity)

Annotation	A cosa serve
`@Id`	Indica la chiave primaria della tabella
`@GeneratedValue(strategy = GenerationType.IDENTITY)`	Il valore dell'ID viene generato automaticamente dal DB
`@Column(nullable = false)`	Il campo NON può essere NULL
`@Column(unique = true)`	Il valore deve essere unico (usato su `title` in Course)
`@JsonIgnore`	NON serializzare questo campo in JSON — evita la ricorsione infinita

Annotation di Metodo (Controller)

Annotation	A cosa serve
`@GetMapping`	Mappa richieste HTTP GET
`@PostMapping`	Mappa richieste HTTP POST
`@PutMapping`	Mappa richieste HTTP PUT
`@DeleteMapping`	Mappa richieste HTTP DELETE
`@PathVariable`	Estrae un valore dall'URL (es. /{id} → Long id)
`@RequestBody`	Deserializza il body JSON della richiesta in un oggetto Java

---

🔗 Focus su `@ManyToMany` e `@JoinTable`

Il Problema Concettuale

Uno studente può essere iscritto a più corsi, e un corso può avere più studenti. Questa è una relazione **molti-a-molti** (many-to-many).

Nel database relazionale non si può rappresentare direttamente una relazione molti-a-molti: serve una **tabella di join** (o tabella intermedia) che collega i due ID. Ad esempio, la tabella `students_course` contiene coppie di (student_id, course_id).

Come è Implementata nel Nostro Codice

Nel progetto, `Student` è il **lato proprietario** (owner) della relazione — è lui che definisce la tabella di join:

```
// In Student.java
@ManyToMany
@JoinTable(
name = "students_course",
joinColumns = @JoinColumn(name = "student_id"),
inverseJoinColumns = @JoinColumn(name = "course_id")
)
@JsonIgnore
private Set<Course> courses = new HashSet<>();
```

`Course` è il **lato inverso** (non-owning) e usa `mappedBy`:

```
// In Course.java
@ManyToMany(mappedBy = "courses")
@JsonIgnore
private Set<Student> students = new HashSet<>();
```

Cosa Genera nel Database

JPA crea automaticamente **tre tabelle**:

Tabella	Contenuto
`students`	id (PK), `first_name`, `last_name`, `email`
`courses`	id (PK), `title`, `description`
`students_course`	`student_id` (FK), `course_id` (FK)

La tabella `students_course` è generata da `@JoinTable` — senza questa annotation, JPA non saprebbe né il nome della tabella né i nomi delle colonne.

Il Ruolo di `@JoinTable`

L'annotation `@JoinTable` specifica:

- **`name`**: il nome della tabella di join nel database (`students_course`)
- **`joinColumns`**: la colonna che referenzia l'entity proprietaria (`student_id` → `Student`)
- **`inverseJoinColumns`**: la colonna che referenzia l'entity inversa (`course_id` → `Course`)

Senza `@JoinTable`, JPA avrebbe nomi di default poco leggibili. Con `@JoinTable`, abbiamo **controllo totale** sulla struttura del database.

Perché `mappedBy` su Course?

Senza `mappedBy`, JPA penserebbe che esistano **due relazioni distinte** e creerebbe **due tabelle di join diverse**. Con `mappedBy = "courses"` si dice esplicitamente: *"questa è la stessa relazione, già mappata nel campo courses di Student"*.

Questo evita duplicazione e inconsistenza nel database.

I Metodi di Utilità in `Student`

Il codice include due metodi fondamentali per gestire correttamente **entrambi i lati** della relazione:

```
public void addCourse(Course course) {
this.courses.add(course); // aggiunge il corso allo studente
course.getStudents().add(this); // aggiunge lo studente al corso
}

public void removeCourse(Course course) {
this.courses.remove(course);
course.getStudents().remove(this);
}
```

Perché è critico: in una relazione bidirezionale, bisogna sempre sincronizzare entrambi i lati in memoria. Se si aggiungesse il corso solo da un lato, l'altro lato sarebbe "stale" (non aggiornato) fino al prossimo caricamento dal DB.

Il Flusso Completo di `enrollStudentInCourse`

Ecco il percorso di una richiesta `POST /api/students/1/enroll/2` (iscrivi studente 1 al corso 2):

1. **Controller riceve** la richiesta HTTP e chiama `studentService.enrollStudentInCourse(1L, 2L)`
2. **StudentService carica** lo Student con id=1 dal database via `studentRepository.findById(1L)`
3. **StudentService carica** il `Course` con id=2 dal database via `courseRepository.findById(2L)`

4. **Chiama** student.addCourse(course) — che aggiorna:
    - student.courses.add(course) — aggiunge il corso al Set dello studente
    - course.students.add(student) — aggiunge lo studente al Set del corso
5. **Salva** con studentRepository.save(student) — Hibernate rileva il cambiamento nel Set<Course> e **inserisce automaticamente una riga nella tabella students_course**
6. **Controller restituisce** lo studente aggiornato come JSON

Esempio di Richiesta e Risposta

Richiesta:
POST /api/students/1/enroll/2

Risposta (JSON):

```
{
"id": 1,
"firstName": "John",
"lastName": "Doe",
"email": "john@example.com",
"courses": [
{
"id": 2,
"title": "Java Advanced",
"description": "Advanced Java Programming"
}
]
}
```


Nota: il campo courses è presente perché @JsonIgnore NON è applicato alla risposta del controller — viene applicato solo durante il serialization di Course per evitare la ricorsione infinita.

Configurazione del Database H2

Il file application.properties contiene le configurazioni per il database H2:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.h2.console.enabled=true
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
```

- `jdbc:h2:mem:testdb` — crea un database H2 in memoria (temporaneo)
- `ddl-auto=create-drop` — crea le tabelle al startup e le elimina allo shutdown

---

Riepilogo degli Annotation Chiave

Annotation	Dove	Perché
`@Entity`	Classe	Trasforma la classe in tabella JPA
`@ManyToMany`	Campo	Dichiara una relazione molti-a-molti
`@JoinTable`	Campo	Specifica la tabella di join nel DB
`@RestController`	Classe	Espone i metodi come endpoint REST
`@Service`	Classe	Contiene la logica di business
`@Transactional`	Metodo/Classe	Gestisce le transazioni automaticamente
`@JsonIgnore`	Campo	Evita la serializzazione ricorsiva in JSON

Conclusioni

Questa applicazione dimostra:

1. **Come modellare una relazione molti-a-molti** con `@ManyToMany` e `@JoinTable`
2. **Come sincronizzare entrambi i lati** di una relazione bidirezionale

3. **L'importanza della separazione a strati** (Entity, Repository, Service, Controller)
4. **Come Spring Data JPA semplifica l'accesso ai dati** senza scrivere SQL
5. **Come usare le annotation di Jackson** (`@JsonIgnore`) per controllare la serializzazione

Il codice è pulito, ben commentato e segue le best practice di Spring Boot moderno.